

UNIVERSIDAD DEL VALLE DE GUATEMALA

COMPUTACIÓN PARALELA Y DISTRIBUIDA - 40

Marlon Fuentes



Proyecto 1. Aurora Paralela

Ihan Marroquin - 23108

Diego Patzan - 23525

Luis Gonzalez - 23343

GUATEMALA, 09 de septiembre del 2026

Índice

Informe final	3
1. Introducción	3
2. Antecedentes	3
3. Diseño y método PCAM.....	4
4. Implementación.....	5
5. Resultados y análisis	6
6. Conclusiones y recomendaciones	8
Referencias bibliográficas.....	9
Anexo 1. Diagrama de flujo.....	10
Anexo 2. Catálogo de funciones	11
Anexo 3. Bitácora de pruebas	14

Informe final

https://github.com/Ihan-Marroquin/Proyecto1_paralela

1. Introducción

Este proyecto desarrolla un screensaver de auroras en C++ con tres rutas de renderizado: secuencial, paralela básica y paralela optimizada. La escena contiene N fuentes luminosas móviles, colores generados de forma reproducible, rebotes contra los bordes y ondas calculadas con funciones trigonométricas. El objetivo práctico fue conservar exactamente la misma imagen en las tres versiones y reducir el tiempo necesario para producir cada cuadro.

La aplicación acepta parámetros de línea de comandos, valida rangos antes de reservar memoria y puede trabajar en una ventana Win32, exportar una imagen PPM, ejecutar pruebas automáticas o medir rendimiento. Esta separación permitió comparar implementaciones sin mezclar el costo del dibujo en pantalla con el cálculo del framebuffer.

2. Antecedentes

OpenMP agrega paralelismo de memoria compartida mediante directivas del compilador y una biblioteca de ejecución. Su modelo permite comenzar con un programa secuencial e introducir regiones paralelas de forma gradual, conservando una sola memoria de proceso [1]. Para este problema, cada píxel se calcula de forma independiente a partir de un estado de fuentes que permanece de solo lectura durante el renderizado.

El speedup se calculó como $S = T_{seq}/T_p$ y la eficiencia como $E = S/p$, donde p es el número de hilos. Estas métricas permiten distinguir entre una reducción real del tiempo y una utilización incompleta de los recursos [2], [3]. La ganancia máxima también queda limitada por preparación, movimiento de datos y otras regiones no paralelas, conforme a la observación de Amdahl [4].

Objetivos

- Construir una versión secuencial correcta y reproducible del screensaver.
- Paralelizar el renderizado con OpenMP sin introducir condiciones de carrera.
- Aplicar una optimización adicional y demostrar su efecto con al menos diez mediciones por versión.

3. Diseño y método PCAM

La arquitectura separa argumentos, simulación, cálculo de color, renderizado, benchmark, pruebas y presentación. AuroraSimulation actualiza el estado físico; AuroraRenderer transforma ese estado en un arreglo de píxeles; los módulos de benchmark y ventana consumen el mismo framebuffer. La misma semilla genera las mismas fuentes, por lo que una comparación de checksums identifica cualquier diferencia entre versiones.

3.1 Partición

El dominio se divide por píxeles. En la versión básica se asignan filas completas; en la optimizada se divide el plano bidimensional con collapse(2). Cada iteración acumula la contribución de las N fuentes sobre un único píxel.

3.2 Comunicación

Los hilos leen el mismo arreglo inmutable de fuentes y escriben posiciones distintas del framebuffer. No hay comunicación dentro del cálculo de un píxel ni actualizaciones compartidas acumulativas.

3.3 Aglomeración

El trabajo por píxel se conserva como una unidad compacta para aprovechar caché y evitar tareas demasiado pequeñas. La versión optimizada precalcula separaciones horizontales y verticales, reduciendo operaciones repetidas dentro del lazo de fuentes.

3.4 Mapeo

OpenMP distribuye las iteraciones con schedule(static). El costo por píxel es aproximadamente uniforme porque todas las posiciones recorren N fuentes; por ello, el reparto estático evita la sobrecarga de una cola dinámica.

3.5 Sincronización y seguridad

La versión básica usa la barrera implícita de omp parallel for. La optimizada mantiene una sola región omp parallel, prepara arreglos auxiliares con dos omp for nowait, ejecuta una barrera explícita y luego renderiza con omp for collapse(2) schedule(static, 256). Ningún hilo escribe en el mismo píxel que otro.

4. Implementación

4.1 Entrada y programación defensiva

N es obligatorio y debe estar entre 1 y 512. También se validan ancho, alto, semilla, cantidad de cuadros, repeticiones y número de hilos. El canvas usa 960x540 de forma predeterminada y no permite dimensiones menores que 640x480. Ante una opción desconocida o un valor inválido se muestra ayuda y se detiene la ejecución sin continuar con un estado parcial.

4.2 Simulación y física

Cada fuente guarda posición, velocidad, fase, amplitud, frecuencia y tono. En cada paso se actualizan posición y fase; al tocar un borde se limita la coordenada y se invierte la componente correspondiente de velocidad. El color final combina distancia, bandas senoidales, modulación temporal y mapeo de tono.

4.3 Versión secuencial

Dos lazos anidados recorren y y x. calculatePixel evalúa las N contribuciones y escribe un valor RGB empaquetado. Esta implementación sirve como referencia funcional y temporal.

4.4 Versión paralela básica

La directiva #pragma omp parallel for schedule(static) reparte las filas. Las variables temporales pertenecen a cada iteración y el estado de fuentes solo se consulta.

4.5 Versión optimizada

Se reutiliza una sola región paralela, se precaculan diferencias en X y Y y se colapsan los dos lazos espaciales. Un bloque de 256 iteraciones conserva un reparto estable y reduce el costo de planificación. Esta ruta obtuvo el menor tiempo promedio.

4.6 Salida

La interfaz Win32 copia el framebuffer a la ventana mediante GDI, permite alternar pantalla completa y atiende los eventos de cierre. En modo headless puede guardar el cuadro como PPM; en benchmark registra tiempo, checksum y un resumen CSV.

5. Resultados y análisis

La siguiente imagen corresponde al cuadro conservado como evidencia del funcionamiento visual. Presenta múltiples colores, variación espacial, movimiento derivado de las fuentes y un canvas de 960x540, superior al mínimo solicitado.

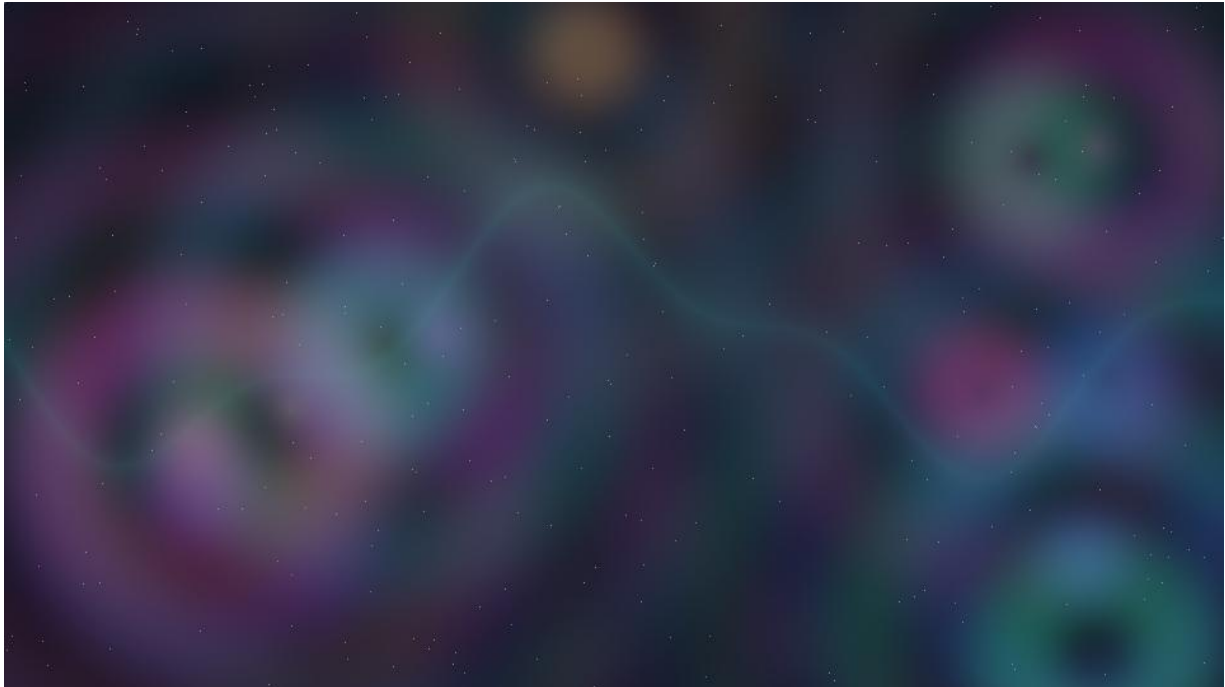


Figura 1. Fondo generado por Aurora Paralela con 16 fuentes y semilla reproducible.

```
PS> .\aurora_saver.exe 16 --benchmark --trials 15 --threads 8
Benchmark: 15 mediciones por modo, 8 hilos
```

Modo	Promedio(ms)	Desv(ms)	Speedup	Eficiencia
sequential	245.896	19.732	1.000	100.000%
parallel	44.313	4.978	5.549	69.363%
optimized	38.703	4.570	6.353	79.418%

Checksums iguales en las 45 ejecuciones comparadas.

Figura 2. Resumen de 15 mediciones por versión con 8 hilos.

5.1 Entorno de prueba

Componente	Configuración
Procesador	Intel Core i7-11800H, 8 núcleos y 16 procesadores lógicos
Memoria	16 GB RAM
Sistema	Windows 11 Pro de 64 bits, versión 10.0.26200
Compilador	g++ MSYS2 16.1.0 con C++17 y OpenMP
Carga	N=16, canvas 960x540, 8 hilos, 15 repeticiones, semilla 2026



Figura 3. Tiempo promedio, speedup y eficiencia de las tres implementaciones.

Modo	Promedio (ms)	Desv. (ms)	Speedup	Eficiencia
Secuencial	245.896	19.732	1.000x	100.00%
Paralelo	44.313	4.978	5.549x	69.36%
Optimizado	38.703	4.570	6.353x	79.42%

La versión paralela redujo 81.98% el tiempo promedio frente a la secuencial. La optimizada añadió una reducción de 12.66% respecto de la paralela básica y elevó el speedup de 5.549x a 6.353x. Los checksums fueron idénticos en cada prueba emparejada, por lo que la mejora no cambió el resultado visual.

6. Conclusiones y recomendaciones

6.1 Conclusiones

- La descomposición por píxeles es adecuada: cada salida puede producirse de forma independiente y el estado compartido se mantiene de solo lectura durante el renderizado.
- OpenMP disminuyó el tiempo promedio de 245.896 ms a 44.313 ms con 8 hilos, para un speedup de 5.549x y 69.36% de eficiencia.
- El precálculo y la región paralela única redujeron el promedio a 38.703 ms, alcanzando 6.353x de speedup y 79.42% de eficiencia.
- La igualdad de checksums y las pruebas píxel por píxel muestran que las optimizaciones conservaron la corrección y reproducibilidad del programa.

6.2 Recomendaciones

- Medir por separado actualización, renderizado y presentación para detectar con más precisión el siguiente cuello de botella.
- Probar varios valores de N y cantidades de hilos; así puede observarse cuándo la sobrecarga paralela deja de compensar.
- Explorar vectorización SIMD y estructuras de datos orientadas a arreglos, manteniendo primero una prueba de equivalencia con la versión secuencial.

6.3 Reproducibilidad

El repositorio incluye CMakeLists.txt, código fuente, encabezados y scripts de compilación, benchmark y pruebas. Los resultados de este informe se obtuvieron con `--seed 2026`, `--threads 8` y `--trials 15`. La ejecución del self-test valida posiciones finitas dentro del canvas, igualdad píxel por píxel, checksums y repetición exacta de una escena con la misma semilla.

Referencias bibliográficas

- [1] OpenMP Architecture Review Board. OpenMP Application Programming Interface, versión 5.2. Noviembre de 2021. <https://www.openmp.org/spec-html/5.2/openmp.html>
- [2] I. Foster. Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering. Addison-Wesley, 1995.
- [3] A. Grama, A. Gupta, G. Karypis e V. Kumar. Introduction to Parallel Computing, segunda edición. Addison-Wesley, 2003.
- [4] G. M. Amdahl. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. AFIPS Spring Joint Computer Conference, pp. 483-485, 1967. <https://doi.org/10.1145/1465482.1465560>

Nota sobre citas

Las afirmaciones técnicas del apartado de antecedentes se sintetizaron a partir de las fuentes [1]-[4]. No se incluyeron citas textuales extensas; las definiciones y fórmulas se expresaron con redacción propia.

Anexo 1. Diagrama de flujo del programa

El flujo incluye captura de argumentos, validación defensiva, inicialización, actualización física, selección de la sección de renderizado, sincronización y presentación o medición de resultados.

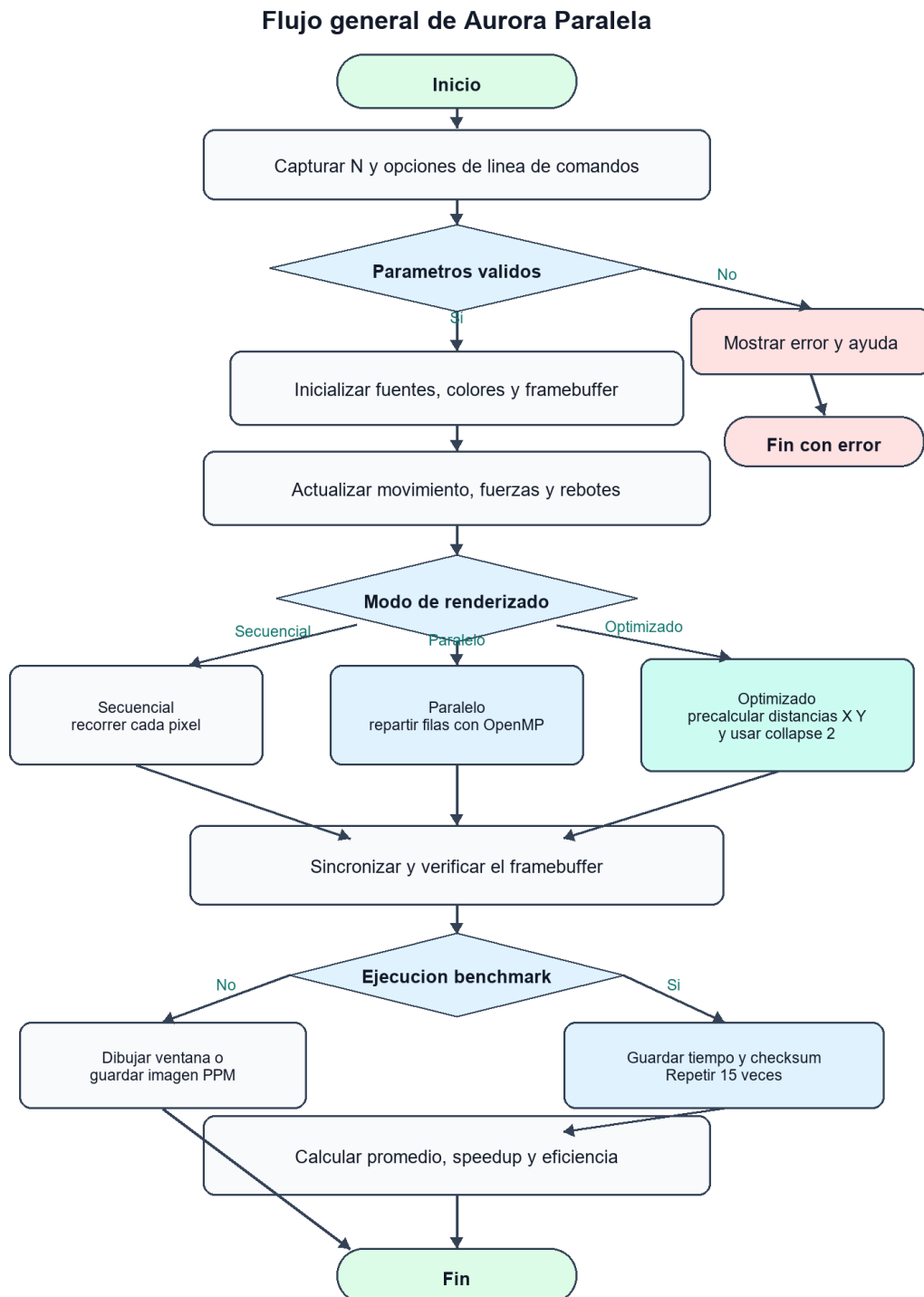


Figura 4. Diagrama de flujo general de Aurora Paralela.

Anexo 2. Catálogo de funciones (1 de 3)

Entradas y salidas usan los tipos declarados en el código. En métodos de clase, el estado del objeto actúa como entrada o salida implícita cuando corresponde.

Función / clase	Entradas	Salida	Descripción
parseInteger	text:string, name:string, min:int, max:int	int	Convierte un argumento decimal y verifica que esté dentro del rango permitido.
parseSeed	text:string	uint32_t	Convierte y valida la semilla usada para reproducir la escena.
renderModeName	mode:RenderMode	string_view	Devuelve el nombre legible del modo de renderizado.
parseArguments	argc:int, argv:char**	ProgramOptions	Procesa N y las opciones; rechaza valores incompletos o desconocidos.
printUsage	stream:ostream	void	Imprime sintaxis, modos y parámetros disponibles.
hueToRgb	hue:float	Color	Convierte un tono normalizado a componentes RGB.
AuroraSimulation	width:int, height:int, count:int, seed:uint32_t	objeto	Reserva el estado de la simulación y crea sus fuentes.
initializeSources	count:int, seed:uint32_t	void	Genera posiciones, velocidades, colores y fases pseudoaleatorias.
update	deltaSeconds:float	void	Avanza posición y fase; aplica rebotes en los límites.
reset	seed:uint32_t	void	Reconstruye la escena con una semilla determinada.
sources	ninguna	const vector<Source>&	Expone las fuentes como vista de solo lectura.
width / height	ninguna	int	Devuelve las dimensiones vigentes de la simulación.

Anexo 2. Catálogo de funciones (2 de 3)

Entradas y salidas usan los tipos declarados en el código. En métodos de clase, el estado del objeto actúa como entrada o salida implícita cuando corresponde.

Función / clase	Entradas	Salida	Descripción
clampUnit	value:float	float	Limita un valor al intervalo [0,1].
toByte	value:float	uint8_t	Convierte intensidad normalizada a un canal de 8 bits.
packColor	r,g,b:uint8_t	uint32_t	Empaqueta tres canales en el formato de píxel del framebuffer.
toneMap	value:float	float	Comprime intensidad para evitar saturación abrupta.
coordinateHash	x:int, y:int, seed:uint32_t	uint32_t	Genera ruido determinista para estrellas y textura.
calculatePixel	x,y:int, sources:span, time:float	uint32_t	Acumula distancias, ondas y color de todas las fuentes.
AuroraRenderer	width:int, height:int	objeto	Reserva el framebuffer y arreglos auxiliares.
render	simulation:const&, mode:RenderMode, threads:int	void	Selecciona la versión secuencial, paralela u optimizada.
renderSequential	simulation:const&	void	Calcula el framebuffer píxel por píxel en un solo hilo.
renderParallel	simulation:const&, threads:int	void	Distribuye filas mediante omp parallel for static.
renderOptimized	simulation:const&, threads:int	void	Precalcula distancias y usa una región paralela con collapse(2).
savePpm	path:filesystem::path	bool	Guarda el framebuffer en un archivo PPM binario.
checksum	ninguna	uint64_t	Calcula una huella del framebuffer para verificar equivalencia.
pixels	ninguna	const vector<uint32_t>&	Expone los píxeles terminados como vista de solo lectura.
width / height	ninguna	int	Devuelve las dimensiones internas del renderer.

Anexo 2. Catálogo de funciones (3 de 3)

Entradas y salidas usan los tipos declarados en el código. En métodos de clase, el estado del objeto actúa como entrada o salida implícita cuando corresponde.

Función / clase	Entradas	Salida	Descripción
summarize	samples:vector<double>, baseline:double, threads:int	BenchmarkSummary	Calcula promedio, mínimo, máximo, desviación, speedup y eficiencia.
prepareCsv	path:filesystem::path	ofstream	Abre el CSV y escribe encabezados cuando corresponde.
runBenchmark	options:ProgramOptions	int	Ejecuta los tres modos, alterna el orden y registra tiempos y checksums.
sourceIsValid	source:const Source&, width:int, height:int	bool	Comprueba valores finitos y posición dentro del canvas.
runSelfTests	ninguna	int	Ejecuta pruebas de física, equivalencia y reproducibilidad.
windowProcedure	hwnd,msg,wParam,lParam	LRESULT	Atiende mensajes de ventana, teclado, pintura y cierre.
drawFramebuffer	hwnd:HWND, renderer:const&	void	Presenta los píxeles en la ventana con GDI.
toggleFullscreen	hwnd:HWND	void	Alternar entre ventana y pantalla completa.
runSaverWindow	options:const ProgramOptions&	int	Crea la ventana y ejecuta el ciclo interactivo.
runHeadless	options:const ProgramOptions&	int	Renderiza sin ventana y opcionalmente guarda una imagen.
main	argc:int, argv:char**	int	Coordina parseo, pruebas, benchmark, salida headless o ventana.

Anexo 3. Bitácora de pruebas (1 de 3)

Se realizaron 15 mediciones por versión. En cada prueba se utilizó el mismo estado para los tres modos; el orden de ejecución se alternó para reducir el sesgo de calentamiento. El cronómetro cubrió únicamente la producción del framebuffer.

```
PS> .\aurora_saver.exe 16 --benchmark --trials 15 --threads 8
Benchmark: 15 mediciones por modo, 8 hilos

Modo          Promedio(ms)  Desv(ms)  Speedup  Eficiencia
sequential    245.896      19.732    1.000    100.000%
parallel      44.313       4.978    5.549    69.363%
optimized     38.703       4.570    6.353    79.418%

Checksums iguales en las 45 ejecuciones comparadas.
```

Figura 5. Captura del resumen generado por el benchmark.

Modo	Promedio	Mínimo	Máximo	Desv.	Speedup	Eficiencia
Secuencial	245.8963	212.0537	275.6745	19.7320	1.0000	100.00%
Paralelo	44.3134	37.5670	53.1205	4.9783	5.5490	69.36%
Optimizado	38.7029	32.8207	45.1687	4.5703	6.3534	79.42%

Los 45 registros se conservaron sin descartar valores. Para una misma prueba, el checksum fue igual en los tres modos.

Anexo 3. Bitácora de pruebas (2 de 3)

Mediciones secuenciales

Prueba	Tiempo (ms)	Checksum
1	235.7974	4441291984371473789
2	240.9693	17493445541569336976
3	247.1784	9203555227680629625
4	230.8778	4289808067906305199
5	231.3214	17547206889561378763
6	221.2160	2591274980435991208
7	212.0537	11682078975811815706
8	224.9986	10766520385134376524
9	242.2948	1548322266691258542
10	270.6323	7707123204654549739
11	270.9512	5168598655018402574
12	275.6745	12876032001457778591
13	253.1357	1020349355258337171
14	273.0667	10105478740328169375
15	258.2761	15518733960695891376

Mediciones paralelas

Prueba	Tiempo (ms)	Checksum
1	38.9365	4441291984371473789
2	42.3418	17493445541569336976
3	42.2530	9203555227680629625
4	39.4367	4289808067906305199
5	40.7232	17547206889561378763
6	37.5670	2591274980435991208
7	39.5422	11682078975811815706
8	39.8231	10766520385134376524
9	51.1921	1548322266691258542
10	53.1205	7707123204654549739
11	50.6639	5168598655018402574
12	49.0911	12876032001457778591
13	47.0106	1020349355258337171
14	48.0360	10105478740328169375
15	44.9632	15518733960695891376

Anexo 3. Bitácora de pruebas (3 de 3)

Mediciones optimizadas

Prueba	Tiempo (ms)	Checksum
1	36.1164	4441291984371473789
2	36.7197	17493445541569336976
3	34.7218	9203555227680629625
4	35.5680	4289808067906305199
5	37.4349	17547206889561378763
6	34.2286	2591274980435991208
7	34.5035	11682078975811815706
8	32.8207	10766520385134376524
9	33.8937	1548322266691258542
10	44.9320	7707123204654549739
11	45.1687	5168598655018402574
12	45.1400	12876032001457778591
13	43.1775	1020349355258337171
14	42.9857	10105478740328169375
15	43.1319	15518733960695891376

```
PS> .\scripts\test.ps1
[OK] las fuentes permanecen dentro del canvas
[OK] el render paralelo coincide pixel por pixel
[OK] el checksum paralelo coincide con la referencia
[OK] el render optimizado coincide pixel por pixel
[OK] el checksum optimizado coincide con la referencia
[OK] la misma semilla reproduce exactamente la escena

Todas las pruebas internas terminaron correctamente.
```

Figura 6. Captura de las pruebas automáticas de corrección.

Resultado: todas las pruebas internas finalizaron correctamente. La comparación píxel por píxel y el checksum detectan errores de sincronización o cambios numéricos entre las implementaciones.